



## Unidade 1 — Back-End e Front-End



## Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Identificar** as principais boas práticas de Front-End;
- **Identificar** as principais boas práticas de Back-End;
- **Aplicar** convenções de nomenclatura e organização de código;
- **Avaliar** a qualidade de um código segundo critérios profissionais;
- **Comparar** código bem escrito com código mal escrito.

**Taxonomia de Bloom:** Identificar, Aplicar, Avaliar, Comparar.



## Por que Boas Práticas Importam?

- Código é **lido** muito mais vezes do que é **escrito**;
- Equipes trabalham **colaborativamente** — código precisa ser compreensível;
- Manutenção consome **mais tempo** do que desenvolvimento inicial;
- Código mal escrito **custa dinheiro** e **tempo**;
- Boas práticas **reduzem bugs** e **facilitam depuração**.

**Dado real:** Estudos indicam que ~70% do tempo de um desenvolvedor é gasto **lendo** código, não escrevendo.



## Boas Práticas — Front-End

| Prática                        | Descrição                                                   |
|--------------------------------|-------------------------------------------------------------|
| HTML semântico                 | Usar tags que comunicam significado                         |
| Separação de responsabilidades | HTML para estrutura, CSS para estilo, JS para comportamento |
| Nomes descritivos              | Classes, IDs e variáveis com nomes claros                   |
| Mobile First                   | Projetar primeiro para mobile, depois para desktop          |
| Acessibilidade                 | ARIA labels, contraste, navegação por teclado               |
| Performance                    | Otimizar imagens, minificar CSS/JS, lazy loading            |



## Boas Práticas — Back-End

| Prática                               | Descrição                                            |
|---------------------------------------|------------------------------------------------------|
| <b>Separação de responsabilidades</b> | Router, Controller, Service, Repository              |
| <b>Validação de dados</b>             | Sempre validar no servidor, nunca confiar no cliente |
| <b>Tratamento de erros</b>            | try/catch, middlewares de erro, mensagens claras     |
| <b>Variáveis de ambiente</b>          | Nunca hardcodar senhas, chaves ou URLs               |
| <b>Versionamento de API</b>           | /api/v1/usuarios, /api/v2/usuarios                   |
| <b>Logging</b>                        | Registrar eventos importantes para depuração         |



## Organização de Projeto

```
projeto/
├── frontend/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── app.js
│   └── assets/
│       └── images/
├── backend/
│   ├── server.js
│   ├── routes/
│   │   └── usuarios.js
│   ├── controllers/
│   │   └── usuarioController.js
│   ├── services/
│   │   └── usuarioService.js
│   └── repositories/
│       └── usuarioRepository.js
├── .env
├── .gitignore
└── README.md
```





## Código Limpo vs. Código Sujo

Código sujo:

```
function x(a, b) {  
  let c = [];  
  for (let i = 0; i < a.length; i++) {  
    if (a[i].s === b) c.push(a[i]);  
  }  
  return c;  
}
```



## Código Limpo vs. Código Sujo

### Código limpo:

```
function filtrarUsuariosPorStatus(usuarios, status) {  
  return usuarios.filter(usuario => usuario.status === status);  
}
```

- Nomes **descritivos**;
- Funções **pequenas** e **focadas**;
- Código **autoexplicativo**.





## Erros Comuns de Iniciantes

| Erro                             | Impacto                                     | Correção                     |
|----------------------------------|---------------------------------------------|------------------------------|
| Usar <code>div</code> para tudo  | Semântica ruim, acessibilidade comprometida | Usar tags semânticas         |
| Misturar HTML, CSS e JS          | Código difícil de manter                    | Separar em arquivos          |
| Hardcodar valores                | Inflexibilidade                             | Usar variáveis de ambiente   |
| Ignorar erros do fetch           | Bugs silenciosos                            | Tratar todos os erros        |
| Não validar dados no servidor    | Falha de segurança                          | Validar sempre no Back-End   |
| Comitar <code>.env</code> no Git | Credenciais expostas                        | Usar <code>.gitignore</code> |



## Síntese da Aula

### Conceitos principais:

1. Código é **lido** mais do que escrito — priorize legibilidade;
2. **Separação de responsabilidades** é fundamental em Front-End e Back-End;
3. **Validação** deve acontecer em ambas as camadas, mas o servidor é o último bastião;
4. **Organização de projeto** facilita navegação e manutenção;
5. **Código limpo** é código que outros desenvolvedores conseguem entender.

### Pergunta reflexiva:

Se você fosse entrar em um projeto existente, o que olharia primeiro para avaliar a qualidade do código?

**Próxima aula:** Aula ao vivo — Integração Front-End + Back-End + API — Vamos colocar tudo em prática!

